



哈尔滨工业大学 法国里昂商学院
Harbin Institute of Technology emlyon business school

Object Oriented Programming Python BootCamp

实战案例合集 / Exam Practice

黄子扬

Ziyang Huang

二叉苹果树

June 13, 2026

Contents

【Example / 实战案例】 Basic Assignment / 基础赋值

```
1 age = 25
2 print(age)          # Output: 25
```

【Example / 实战案例】 Valid and Invalid Names / 合法与非法命名

```
1 # Valid names / 合法命名
2 test1 = 1
3 _alpha = 2
4 L = 3
5 car_length = 2000
6
7 # Invalid names / 非法命名
8 # 3text = 4          # Error: cannot start with digit
9 # my-var = 5         # Error: hyphen is not allowed
```

【Example / 实战案例】 Comment Examples / 注释示例

```
1 # This variable stores the car length in cm
2 L = 2000
3 print(L)
4
5 H = 146   # This variable stores the car height in cm
6 print(H)
```

【Example / 实战案例】 Using type() / type() 实操

```
1 num = 100          # An integer variable
2 length = 10.7      # A floating point variable
3
4 print(type(num))    # <class 'int'>
5 print(type(length)) # <class 'float'>
```

【Example / 实战案例】 Boolean Examples / 布尔值示例

```
1 is_finished = True
2 print(type(is_finished)) # <class 'bool'>
3
4 is_finished = False
5 print(type(is_finished)) # <class 'bool'>
```

【Example / 实战案例】Escape Character / 转义字符示例

```
1 text1 = 'She said, "What time is it?'"
2 text2 = "She said, \"What time is it?\""
3 print(text1)      # She said, "What time is it?"
4 print(text2)      # She said, "What time is it?"
```

【Example / 实战案例】Using len() / len() 函数

len() is a Python built-in function that returns the number of characters in a string (including spaces).

```
1 text2 = "This is a test string"
2 print(len(text2))  # Output: 22
```

【Example / 实战案例】Multiline String Examples / 多行字符串示例

```
1 # Method 1: Backslash / 反斜杠法
2 long_text = "This planet has--or rather had--a problem, which was \
3 this: most of the people living on it were unhappy for pretty much of the time."
4 print(long_text)
5
6 # Method 2: Triple quotes / 三引号法
7 long_text = """This planet has--or rather had--a problem, which was
8 this: most of the people living on it were unhappy for pretty much of the time."""
9 print(long_text)
```

【Example / 实战案例】Concatenation Example / 拼接示例

```
1 string1 = "Hello"
2 string2 = "World"
3 string3 = string1 + string2
4 print(string3)      # Output: HelloWorld (no space!)
```

【Example / 实战案例】Positive and Negative Indexing / 正索引与负索引

```
1 text = "Python BootCamp"
2
3 # Positive index / 正索引 (从左到右, 从 0 开始)
4 print(text[0])      # 'P' (第一个字符)
5 print(text[7])      # 'B'
6
7 # Negative index / 负索引 (从右到左, 从 -1 开始)
8 print(text[-1])     # 'p' (最后一个字符)
9 print(text[-5])     # 't'
```

【Example / 实战案例】Slicing Examples / 切片实战

```
1 text = "Python BootCamp"
2
3 # Basic slicing [start:stop]
4 print(text[0:6])      # 'Python' (indices 0,1,2,3,4,5 — NOT 6)
5 print(text[7:11])     # 'Boot'
6
7 # Omit start: assumes 0
8 print(text[:6])       # 'Python' (same as text[0:6])
9
10 # Omit stop: goes to the end
11 print(text[7:])       # 'BootCamp'
12
13 # Both omitted: the whole string
14 print(text[:])        # 'Python BootCamp'
```

【Example / 实战案例】Immutability Demonstration / 不可变性演示

```
1 name = "Emlyon"
2 name.upper()          # Returns "EMLYON" but does NOT change 'name'!
3 print(name)           # Still "Emlyon"
4
5 name = name.upper()    # Correct: reassign the result
6 print(name)           # Now "EMLYON"
```

【Example / 实战案例】Common String Methods / 常用字符串方法

```
1 # 1. Case Conversion / 大小写转换
2 print("Python BootCamp".lower())      # 'python bootcamp'
3 print("Python BootCamp".upper())      # 'PYTHON BOOTCAMP'
4
5 # 2. Removing Whitespace / 去除首尾空白
6 print(" Python BootCamp".strip())     # 'Python BootCamp'
7
8 # 3. Counting Substrings / 统计子串出现次数
9 text = "Python BootCamp is the best Python program ever!"
10 print(text.count("Python"))           # Output: 2
11
12 # 4. Finding Substring Position / 查找子串位置
13 print(text.find("Python"))             # Output: 0 (first occurrence)
14 print(text.find("python"))             # Output: -1 (not found!)
15
16 # 5. Check Start and End / 检查首尾
17 print("Python BootCamp".startswith('P')) # True
18 print("Python BootCamp".endswith('p'))   # False
19
20 # 6. Replace / 替换
21 print("Hello World".replace("World", "Python")) # 'Hello Python'
```

【Example / 实战案例】None Example / None 示例

```
1 my_var = None
2 print(my_var)          # None
3 print(type(my_var))    # <class 'NoneType'>
```

【Example / 实战案例】Arithmetic Operators Demo / 算术运算符演示

```
1 print(2 + 2)           # 4
2 print(3 * 2)           # 6
3 print(3 ** 2)          # 9
4 print(5 / 2)           # 2.5 (standard division)
5 print(5 // 2)          # 2 (floor division)
```

【Example / 实战案例】String Arithmetic / 字符串运算

```
1 print('2' + '4')       # '24' (concatenation, not addition!)
2 print(2 * 'S')          # 'SS' (repetition)
3 # print('2' * '4')      # Error! Cannot multiply string by string
```

【Example / 实战案例】Comparison Examples / 比较运算示例

```
1 print(2.5 < 3)          # True
2 print(2 == 2)           # True
3 print(2 != 3)           # True
4 print(5 >= 5)           # True
5 print(10 < 3)           # False
```

【Example / 实战案例】Logical Operators Demo / 逻辑运算符演示

```
1 # or: True if either is True
2 print((2 == 3) or (4 < 5))    # True (4 < 5 is True)
3
4 # and: True only if BOTH are True
5 print((2 == 3) and (4 < 5))   # False (2 == 3 is False)
6
7 # not: Reverses the boolean
8 print(not (2 == 3))           # True (reverses False to True)
9 print(not True)               # False
```

【Example / 实战案例】Type Casting Examples / 类型转换示例

```
1 print(int(9.9))           # 9 (truncates, does NOT round!)
2 print(str(5))              # '5'
```

```
3 print(bool(1))          # True
4 # print(int("Yes!"))    # ValueError: invalid literal for int()
```

【Example / 实战案例】Comma Method / 逗号法

```
1 name = "Doctor Strange"
2 heads = 2
3 arms = 3
4 print(name, "has", str(heads), "heads and", str(arms), "arms.")
5 # Output: Doctor Strange has 2 heads and 3 arms.
```

Note: `print()` automatically inserts spaces between comma-separated items. You must convert numbers to strings with `str()`.

注意: `print()` 会自动在逗号分隔的项之间插入空格。数字必须用 `str()` 转为字符串。

【Example / 实战案例】Concatenation Method / 拼接法

```
1 print(name + " has " + str(heads) + " heads and " + str(arms) + " arms.")
2 # Output: Doctor Strange has 2 heads and 3 arms.
```

Note: Tedious! Must manually add spaces and convert numbers to strings.

注意: 这种方法写起来很繁琐! 必须手动添加空格、用 `str()` 转换数字。

【Example / 实战案例】f-string Example / f-字符串示例

```
1 print(f"{name} has {heads} heads and {arms} arms.")
2 # Output: Doctor Strange has 2 heads and 3 arms.
```

【Example / 实战案例】.format() Method / .format() 方法示例

```
1 print("{} has {} heads and {} arms.".format(name, heads, arms))
2 # Output: Doctor Strange has 2 heads and 3 arms.
```

Curly braces `{}` act as placeholders, filled by `.format()` arguments in order.

【Example / 实战案例】% Operator / % 运算符示例

```
1 print("%s has %s heads and %s arms." % (name, heads, arms))
2 # Output: Doctor Strange has 2 heads and 3 arms.
```

`%s` is a placeholder for string; values are provided in parentheses after `%`.

【Example / 实战案例】input() Examples / 用户输入示例

```
1 # Basic usage
2 name = input()
```

```
3 print(name)
4
5 # With a prompt / 带提示信息的输入
6 age = input("Please enter your age: ")
7 print(age)                # age is a STRING, not a number!
8 print(type(age))          # <class 'str'>
9
10 # Convert to number if needed / 如需数字, 必须转换
11 age = int(input("Please enter your age: "))
12 print(age + 1)           # Now it works as a number
```

【Example / 实战案例】Price Classification / 价格分级案例

课件中通过以下案例展示了完整的 `if-elif-else` 结构:

```
1 price = 42
2 if price > 100:
3     print("expensive")    # Runs if price > 100
4 elif price > 50:
5     print("moderate")     # Checked ONLY if price <= 100
6 else:
7     print("cheap")        # Runs if all above are False
8 # Output: cheap          # Runs if price <= 50
```

【Example / 实战案例】Simple If-Statement / 简单 If 判断

课件展示了只需判断单个条件的场景:

```
1 name = "John"
2 if len(name) < 3:
3     print("your name is short!")    # Won't run because len("John") = 4
```

如果条件为 `False`, 缩进代码块什么都不执行。

【Example / 实战案例】Understanding len() / 理解 len() 的执行过程

```
1 num_letters = len("emlyon")
2 print(num_letters)    # Output: 6
```

What happens behind the scenes:

1. `len()` is called with the argument `"emlyon"`.
2. The length of the string is calculated —the number 6.
3. `len()` returns 6 and replaces the function call with the value.
4. Python then assigns 6 to `num_letters`. Equivalent to: `num_letters = 6`

【Example / 实战案例】Creating Your First Function / 创建你的第一个函数

```
1 def multiply(x, y):           # Function signature: name + parameters
2     product = x * y          # Function body (indented!)
3     return product            # Return statement: sends result back
4
5 # Calling the function
6 result = multiply(2, 4)       # result = 8
7 print(result)                 # 8
```

【Example / 实战案例】Scope Demonstration / 作用域演示

```
1 x = 2                         # Global x / 全局变量
2
3 def func():
4     x = 3                     # NEW local x — does NOT change global x!
5     print("Inside:", x)      # Prints 3 (local x)
6
7 func()                        # Output: Inside: 3
8 print("Outside:", x)         # Output: 2 (global x unchanged!)
```

关键理解：函数内部的 `x = 3` 创建了一个全新的局部变量，不会影响全局的 `x`。

【Example / 实战案例】Scope Lookup Example / 作用域查找示例

```
1 x = 5                         # Global
2
3 def func():
4     y = x + 5                 # x not found locally → uses global x (5)
5     print("Inside, y =", y)  # Output: Inside, y = 10
6
7 func()
8 print("Outside, x =", x)     # Output: Outside, x = 5
9 # print(y)                   # NameError! y only exists in func()'s local scope
```

注意：函数内部可以读取全局变量，但不能直接修改它（除非用 `global` 关键字）。

【Example / 实战案例】Printing Numbers with for / 用 for 循环打印数字

```
1 for i in range(1, 11):
2     print(i)                  # Prints 1, 2, 3, ..., 10 (each on new line)
```

其中 `i` 是迭代变量，依次取值为 1, 2, 3, ..., 10。

【Example / 实战案例】While Loop vs For Loop / While 循环与 For 循环对比

```
1 # For loop version — finite, known iterations
2 for i in range(1, 11):
3     print(i)
```

```
4
5 # While loop version — same result, different approach
6 counter = 1
7 while counter < 11:
8     print(counter)
9     counter = counter + 1      # MUST update counter!
```

【Example / 实战案例】Control Keywords Demo / 控制关键字对比

```
1 # Example: break — 立即终止整个循环
2 for i in range(1, 11):
3     if i == 5:
4         break          # Exit loop completely when i = 5
5     print(i)           # Prints: 1, 2, 3, 4 (stops at 5)
```

```
1 # Example: continue — 跳过本次循环剩余部分
2 for i in range(1, 11):
3     if i == 5:
4         continue       # Skip the rest of THIS iteration
5     print(i)           # Prints: 1, 2, 3, 4, 6, 7, 8, 9, 10 (no 5!)
```

```
1 # Example: pass — 占位符，什么都不做
2 for i in range(1, 11):
3     if i == 5:
4         pass           # Do nothing — just a placeholder
5     print(i)           # Prints: 1, 2, 3, 4, 5, 6, 7, 8, 9, 10
```

【Example / 实战案例】print() with sep and end / print() 的 sep 和 end 参数

```
1 # Default behavior
2 print("hello", "there")
3 # Output: hello there (separated by space, ends with newline)
4
5 # Custom sep and end
6 print("hello", "there", sep="", end="!")
7 # Output: hellothere! (no separator, ends with "!" instead of newline)
```

【Example / 实战案例】Tuple Basics / 元组基础

```
1 tuple1 = (1, 2, 3)
2 print(type(tuple1))          # <class 'tuple'>
3
4 # Heterogeneous / 异构：可以包含不同类型
5 tuple2 = (1, "John", "Red")
6 print(tuple2)                # (1, 'John', 'Red')
7
8 # Empty tuple / 空元组
```

```
9 tuple3 = ()
10 print(len(tuple3))           # 0
```

【Example / 实战案例】Length, Indexing, Slicing / 长度、索引、切片

```
1 tuple2 = (1, "John", "Red")
2
3 # Length / 长度
4 print(len(tuple2))           # 3
5
6 # Indexing / 索引 (和字符串一样!)
7 print(tuple2[1])             # 'John'
8
9 # Slicing / 切片
10 print(tuple2[1:3])           # ('John', 'Red')
```

【Example / 实战案例】The in Keyword / in 关键字

```
1 tuple2 = (1, "John", "Red")
2
3 if "Red" in tuple2:
4     print('There is red')     # This runs
5 else:
6     print('There is no red')
```

【Example / 实战案例】Tuples Are Iterable / 元组可迭代

```
1 tuple2 = (1, "John", "Red")
2 for item in tuple2:
3     print(item)
4 # Output:
5 # 1
6 # John
7 # Red
```

【Example / 实战案例】Creating Lists / 创建列表

```
1 mylist1 = [1, 2, 3]
2 print(type(mylist1))         # <class 'list'>
3
4 # Like tuples, lists can contain different types
5 mylist2 = [1, "John", "Red"]
6 print(mylist2)               # [1, 'John', 'Red']
7
8 # Create a list from string using split()
9 groceries = "eggs, milk, cheese"
```

```
10 grocery_list = groceries.split(", ")
11 print(grocery_list)           # ['eggs', 'milk', 'cheese']
```

【Example / 实战案例】Mutability Demo / 可变性演示

```
1 grocery_list = ['eggs', 'milk', 'cheese']
2 grocery_list[1] = 'butter'
3 print(grocery_list)           # ['eggs', 'butter', 'cheese']
```

【Example / 实战案例】insert(), pop(), append(), extend() / 四大核心方法

```
1 mylist2 = []
2 mylist2.insert(0, 'A')         # Insert 'A' at index 0
3 print(mylist2)                 # ['A']
4
5 mylist2.insert(2, 'C')         # Insert at index 2 (beyond end → appended)
6 print(mylist2)                 # ['A', 'C']
7
8 mylist2.insert(1, 'B')         # Insert 'B' at index 1
9 print(mylist2)                 # ['A', 'B', 'C']
10
11 # pop(i): Remove and RETURN the value at index i
12 out = mylist2.pop(2)          # Removes 'C'
13 print(out)                    # 'C'
14 print(mylist2)                # ['A', 'B']
15
16 # append(x): Add to the end
17 mylist2.append('D')
18 print(mylist2)                 # ['A', 'B', 'D']
19
20 # extend(iterable): Add multiple elements to the end
21 mylist2.extend(['E', 'F'])
22 print(mylist2)                 # ['A', 'B', 'D', 'E', 'F']
```

【Example / 实战案例】sum(), min(), max() / 求和、最小值、最大值

```
1 mylist3 = [4, 17, 42, 5]
2
3 # Traditional way / 传统方式
4 total = 0
5 for number in mylist3:
6     total = total + number
7 print(total)                   # 68
8
9 # Pythonic way / Python 风格
10 print(sum(mylist3))            # 68
11 print(min(mylist3))            # 4
12 print(max(mylist3))            # 42
```

【Example / 实战案例】Right Way to Copy / 正确拷贝方式

```
1 animals = ["lion", "tiger"]
2
3 # WRONG way / 错误方式 — 只是别名
4 large_cats = animals
5 large_cats.append("Leopard")
6 print(large_cats)    # ['lion', 'tiger', 'Leopard']
7 print(animals)       # ['lion', 'tiger', 'Leopard'] — ALSO changed!
8
9 # RIGHT way / 正确方式 — 使用切片创建独立副本
10 animals = ["lion", "tiger"]
11 large_cats = animals[:]    # Creates a NEW list with same values
12 large_cats.append("Leopard")
13 print(large_cats)    # ['lion', 'tiger', 'Leopard']
14 print(animals)       # ['lion', 'tiger'] — unchanged!
```

【Example / 实战案例】Nested Structures / 嵌套示例

```
1 my_nested_list = [[1, 2], [3, 4]]
2 print(my_nested_list)    # [[1, 2], [3, 4]]
3 print(my_nested_list[1]) # [3, 4]
4
5 # Double index notation / 双重索引
6 print(my_nested_list[1][0]) # 3 (first element of second sublist)
```

【Example / 实战案例】Sorting Examples / 排序示例

```
1 # .sort() — in-place, list only
2 numbers = [1, 10, 5, 3]
3 numbers.sort()    # Modifies 'numbers' directly
4 print(numbers)    # [1, 3, 5, 10]
5
6 # sorted() — returns new list, works on any iterable
7 numbers = [1, 10, 5, 3]
8 new = sorted(numbers, reverse=True)
9 print(new)        # [10, 5, 3, 1]
10 print(numbers)    # [1, 10, 5, 3] — unchanged!
11
12 # Sort by key / 按自定义规则排序
13 colors = ["red", "yellow", "green", "blue"]
14 colors.sort(key=len)    # Sort by string length
15 print(colors)          # ['red', 'blue', 'green', 'yellow']
```

【Example / 实战案例】 Dictionary Basics / 字典基础操作

```
1 # Creating a dictionary / 创建字典
2 capitals = {
3     "France": "Paris",
4     "Spain": "Madrid",
5     "Italy": "Rome",
6 }
7 print(capitals)
8 # {'France': 'Paris', 'Spain': 'Madrid', 'Italy': 'Rome'}
9
10 # Empty dictionary / 空字典 (两种方式)
11 dict_example = {}
12 dict_example = dict()
13
14 # Accessing values / 访问值
15 print(capitals["France"])      # 'Paris'
```

【Example / 实战案例】 Adding and Deleting / 增删操作

```
1 capitals = {"France": "Paris", "Spain": "Madrid"}
2
3 # Adding a new item / 添加新条目
4 capitals["Germany"] = "Berlin"
5 print(capitals)
6 # {'France': 'Paris', 'Spain': 'Madrid', 'Germany': 'Berlin'}
7
8 # Deleting an item / 删除条目
9 del capitals["Spain"]
10 print(capitals)
11 # {'France': 'Paris', 'Germany': 'Berlin'}
```

【Example / 实战案例】 Safe Key Access / 安全的键访问

```
1 capitals = {"France": "Paris", "Spain": "Madrid"}
2
3 # Check before accessing / 先检查再访问
4 if "Spain" in capitals.keys():
5     print(capitals["Spain"])      # 'Madrid'
6
7 # Simpler version / 更简洁的写法
8 if "Spain" in capitals:          # in 默认检查 keys
9     print(capitals["Spain"])
```

【Example / 实战案例】 Dictionary Iteration / 字典遍历方法

```
1 capitals = {"France": "Paris", "Spain": "Madrid", "Italy": "Rome"}
2
```

```
3 # Method 1: Loop over keys (default) / 默认遍历键
4 for key in capitals:
5     print(key)
6 # Output: France, Spain, Italy (order may vary in Python < 3.7)
7
8 # Method 2: Access both key and value / 同时遍历键和值
9 for country in capitals:
10     print(f"The capital of {country} is {capitals[country]}")
11
12 # Method 3: Using .items() — most "Pythonic" / 最 Python 的写法
13 for country, capital in capitals.items():
14     print(f"The capital of {country} is {capital}")
15 # .items() returns (key, value) tuples that can be unpacked
```

【Example / 实战案例】Set Creation / 创建集合

```
1 myset = {"apple", "banana", "cherry"}
2 print(myset)                # {'apple', 'banana', 'cherry'}
3 print(type(myset))          # <class 'set'>
4
5 # Duplicates are automatically removed! / 重复值自动去重
6 myset = {"apple", "banana", "cherry", "apple"}
7 print(myset)                # {'apple', 'banana', 'cherry'} (no duplicate)
```

【Example / 实战案例】Set Operations / 集合运算

集合最大的优势在于数学集合运算——去重、交集、并集、差集等操作一行搞定：

```
1 A = {1, 2, 3, 4}
2 B = {3, 4, 5, 6}
3
4 # Add/Remove elements / 添加与删除
5 A.add(10)                    # {1, 2, 3, 4, 10}
6 A.remove(10)                 # {1, 2, 3, 4} — KeyError if not found
7 A.discard(99)                # Safe remove — no error if missing
8
9 # Union / 并集：A 或 B 中的所有元素
10 print(A | B)                 # {1, 2, 3, 4, 5, 6}
11 print(A.union(B))            # Equivalent
12
13 # Intersection / 交集：同时在 A 和 B 中的元素
14 print(A & B)                  # {3, 4}
15 print(A.intersection(B))     # Equivalent
16
17 # Difference / 差集：在 A 中但不在 B 中的元素
18 print(A - B)                 # {1, 2}
19
20 # Symmetric Difference / 对称差：仅在 A 或仅在 B (不同时存在)
21 print(A ^ B)                 # {1, 2, 5, 6}
22
```

```
23 # Subset & Superset / 子集与超集
24 print({1, 2} <= A)           # True — {1, 2} is subset of A
25 print(A >= {1, 2})          # True — A is superset of {1, 2}
26
27 # Practical use: deduplicate a list / 实战：列表去重
28 duplicates = [1, 2, 2, 3, 3, 3, 4]
29 unique = list(set(duplicates)) # [1, 2, 3, 4] — 一行去重!
```

【Example / 实战案例】Comprehension vs Traditional Loop / 推导式 vs 传统循环

```
1 numbers = (1, 2, 3, 4, 5)
2
3 # Traditional for loop / 传统 for 循环
4 squares = []
5 for num in numbers:
6     squares.append(num**2)
7 print(squares)           # [1, 4, 9, 16, 25]
8
9 # List comprehension / 列表推导式 (一行搞定!)
10 squares = [num**2 for num in numbers]
11 print(squares)          # [1, 4, 9, 16, 25]
```

【Example / 实战案例】Comprehension with Type Conversion / 推导式 + 类型转换

列表推导式常用于将列表中的值转换为不同类型:

```
1 # Convert strings to floats / 字符串列表 → 浮点数列表
2 str_numbers = ["1.5", "2.3", "5.25"]
3 float_numbers = [float(value) for value in str_numbers]
4 print(float_numbers)     # [1.5, 2.3, 5.25]
```

【Example / 实战案例】Filtered & Dict/Set Comprehension / 条件过滤 + 字典/集合推导式

```
1 numbers = [1, 2, 3, 4, 5, 6]
2
3 # List comprehension with condition / 带过滤的列表推导式
4 even_squares = [num**2 for num in numbers if num % 2 == 0]
5 print(even_squares)      # [4, 16, 36]
6
7 # if-else inside comprehension (ternary) / 条件表达式在左侧
8 labels = ["even" if n % 2 == 0 else "odd" for n in numbers]
9 print(labels)            # ['odd', 'even', 'odd', 'even', ...]
10
11 # Dictionary comprehension / 字典推导式
12 squares_dict = {num: num**2 for num in numbers}
13 print(squares_dict)     # {1: 1, 2: 4, 3: 9, 4: 16, ...}
14
15 # Dict comprehension with filter / 带过滤的字典推导式
16 even_squares_dict = {n: n**2 for n in numbers if n % 2 == 0}
```



```
17 print(even_squares_dict)                # {2: 4, 4: 16, 6: 36}
18
19 # Set comprehension / 集合推导式
20 unique_lengths = {len(str(n)) for n in numbers}
21 print(unique_lengths)                    # {1} — all single-digit lengths are 1
22
23 # Nested comprehension / 嵌套推导式 (展开二维列表)
24 matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
25 flattened = [x for row in matrix for x in row]
26 print(flattened)                        # [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

【Example / 实战案例】Import Examples / 导入示例

```
1 # 1. Basic import
2 import numpy
3 arr = numpy.array([1, 2, 3])
4
5 # 2. Import with alias (most common for NumPy/Pandas)
6 import numpy as np
7 arr = np.array([1, 2, 3])
8
9 # 3. Import specific function
10 from math import sqrt
11 print(sqrt(16))                        # 4.0
12
13 # 4. Import specific function with alias
14 from math import sqrt as square_root
15 print(square_root(16))                 # 4.0
```

【Example / 实战案例】Shallow Copy Example / 浅拷贝案例

```
1 import copy
2 animals = ["lion", "tiger"]
3
4 # Without copy — just a reference
5 large_cats = animals
6 large_cats.append("Leopard")
7 print(animals)                        # ['lion', 'tiger', 'Leopard'] — CHANGED!
8
9 # With copy.copy — independent copy
10 animals = ["lion", "tiger"]
11 large_cats = copy.copy(animals)
12 large_cats.append("Leopard")
13 print(large_cats)                    # ['lion', 'tiger', 'Leopard']
14 print(animals)                      # ['lion', 'tiger'] — unchanged!
```

【Example / 实战案例】 Deep Copy Example / 深拷贝案例

```
1 import copy
2
3 # Nested list — shallow copy still shares inner objects
4 original = [[1, 2, 3], [4, 5, 6]]
5 shallow = copy.copy(original)
6 deep = copy.deepcopy(original)
7
8 # Modify inner element of shallow copy
9 shallow[0][0] = 999
10 print(original)          # [[999, 2, 3], [4, 5, 6]] — CHANGED!
11 print(shallow)           # [[999, 2, 3], [4, 5, 6]]
12 print(deep)              # [[1, 2, 3], [4, 5, 6]] — unchanged!
13
14 # Modify inner element of deep copy
15 deep[1][0] = 777
16 print(original)          # [[999, 2, 3], [4, 5, 6]] — unchanged!
17 print(deep)              # [[1, 2, 3], [777, 5, 6]] — independent!
18
19 # Key insight: shallow copy copies the container only;
20 # deep copy recursively copies everything inside.
```

【Example / 实战案例】 math Module Examples / math 模块示例

```
1 import math
2
3 print(math.log(5, 2))      # Logarithm base 2 of 5
4 print(math.pi)            # 3.141592653589793
5 print(math.floor(2.6))    # 2 (round down)
6 print(math.ceil(2.6))     # 3 (round up)
```

【Example / 实战案例】 Time Module Examples / time 模块示例

```
1 import time
2
3 # Get current Unix timestamp / 获取当前时间戳
4 print(time.time())        # e.g., 1717718400.123456
5
6 # Pause execution for 2 seconds / 暂停 2 秒
7 print("Wait...")
8 time.sleep(2)
9 print("Hi")               # Printed after 2 seconds
10
11 # Timing how long a function takes / 测量函数执行时间
12 def my_function():
13     for n in range(1000000):
14         n += 2 * n
15
16 start_time = time.time()
```

```
17 my_function()
18 elapsed = time.time() - start_time
19 print(f"{elapsed:.4f} seconds")
```

【Example / 实战案例】Time Object with localtime() / 本地时间对象

```
1 now = time.localtime()
2 print(now)                                # time.struct_time object
3 print(now.tm_year)                        # Current year, e.g., 2026
4 print(now.tm_mon)                         # Current month
5 print(now.tm_mday)                        # Current day
```

【Example / 实战案例】Common pip Commands / 常用 pip 命令

```
1 # Check pip version / 检查 pip 版本
2 pip --version
3
4 # Upgrade pip itself / 升级 pip 自身
5 pip install --upgrade pip
6
7 # List all installed packages / 列出已安装的所有包
8 pip list
9
10 # Install a package / 安装包
11 pip install numpy
12
13 # Install a specific version / 安装特定版本
14 pip install numpy==1.16.5
15
16 # Uninstall a package / 卸载包
17 pip uninstall numpy
```

【Example / 实战案例】Speed Comparison / 速度对比

NumPy 的向量化运算比纯 Python 列表快几十倍——关键原因是不需要对每个元素逐一进行类型检查。

```
1 import random
2 import numpy as np
3
4 # Create 1,000,000 random integers with Python list / 用 Python 列表创建100万随机整数
5 measurements = [random.randint(150, 200) for _ in range(1_000_000)]
6
7 # NumPy version / 转换为 NumPy 数组
8 measurements_array = np.array(measurements)
9
10 # NumPy's vectorized mean is MUCH faster / 向量化均值快约50倍!
11 np.mean(measurements_array)      # ~50x faster than sum/len on list
```

【Example / 实战案例】Creating Arrays / 创建数组案例

```
1 import numpy as np
2
3 # From list
4 print(np.array([10, 2, 35]))          # [10  2 35]
5
6 # arange: like range but NumPy
7 print(list(range(5)))                 # [0, 1, 2, 3, 4]
8 print(np.arange(start=2, stop=14, step=2)) # [ 2  4  6  8 10 12]
9
10 # linspace: N evenly spaced values
11 print(np.linspace(start=-5, stop=5, num=9))
12 # [-5.  -3.75 -2.5  -1.25  0.   1.25  2.5   3.75  5. ]
13
14 # zeros and ones
15 print(np.zeros(3))                   # [0. 0. 0.]
16 print(np.ones(3))                    # [1. 1. 1.]
17
18 # Multidimensional with shape argument
19 print(np.zeros((2, 2)))               # 2x2 matrix of zeros
20 # [[0. 0.]
21 #  [0. 0.]]
22 print(np.ones((2, 3)))               # 2x3 matrix of ones
```

【Example / 实战案例】Array Anatomy / 数组属性详解

```
1 import numpy as np
2
3 # dtype — data type
4 values = [0, 1, 2, 3, 4]
5 int_arr = np.array(values, dtype='int')
6 print(int_arr, int_arr.dtype)        # [0 1 2 3 4] int64
7
8 # NumPy casts values to match dtype
9 bool_arr = np.array(values, dtype='bool')
10 print(bool_arr, bool_arr.dtype)
11 # [False  True  True  True  True] bool
12
13 # Without explicit dtype: "smallest common denominator"
14 values = [0, 1, 2.5, 3, 4]
15 float_arr = np.array(values)
16 print(float_arr, float_arr.dtype)    # all become float64
17
18 # 1D Array
19 one_dim_arr = np.array([1, 2, 3, 4])
20 print(one_dim_arr.shape)              # (4,)
21 print(one_dim_arr.ndim)              # 1
22
23 # 2D Array
24 values = [[1, 2, 3, 4, 5],
25           [1, 2, 3, 4, 1],
```

```
26         [1, 2, 3, 4, 2]]
27 two_dim_arr = np.array(values)
28 print(two_dim_arr.shape)           # (3, 5)
29 print(two_dim_arr.ndim)           # 2
30
31 # Access 2D elements: [row, column]
32 print(two_dim_arr[1, 3])           # 4
33 two_dim_arr[1, 3] = 10             # Lists are mutable!
34 print(two_dim_arr)
35
36 # 3D Array
37 values = [[[1, 2, 3, 4]] * 3] * 6
38 three_dim_arr = np.array(values)
39 print(three_dim_arr.shape)         # (6, 3, 4)
40 print(three_dim_arr.ndim)         # 3
41 print(three_dim_arr[1, 1, 1])     # Access 3D element
```

【Example / 实战案例】 Using reshape() / reshape() 用法

```
1 a = np.arange(start=2, stop=14)    # [2, 3, ..., 13] — 12 elements
2 print(a.shape)                    # (12,)
3
4 # Reshape to 3x4
5 print(a.reshape(3, 4))
6 # [[ 2  3  4  5]
7 #   [ 6  7  8  9]
8 #   [10 11 12 13]]
9
10 # -1 auto-detects the size of that dimension
11 print(a.reshape(2, -1))           # 2x6
12 print(a.reshape(2, -1).shape)     # (2, 6)
```

【Example / 实战案例】 Equality and np.isclose() / 相等比较与 np.isclose()

```
1 a = np.zeros((3, 3))
2 b = np.zeros((3, 3))
3 print(a == b)                     # All True (element-wise comparison)
4
5 # Floating point precision issue!
6 a[0, 0] += 0.0000000000000001
7 print(a == b)                     # One element is now False!
8
9 # Use np.isclose() for floating point comparison
10 print(np.isclose(a, b))           # All True (tolerates tiny differences)
```

【Example / 实战案例】 Vectorized Operations / 向量化运算

```
1 arr = np.arange(1, 10)           # [1 2 3 4 5 6 7 8 9]
2
3 # Element-wise operations / 逐元素运算
4 print(arr * 3)                   # [ 3  6  9 12 15 18 21 24 27]
5 print(arr * arr)                 # [ 1  4  9 16 25 36 49 64 81]
6 print(arr + (arr * 2))           # [ 3  6  9 12 15 18 21 24 27]
7 print(arr - arr)                 # [0 0 0 0 0 0 0 0 0]
8 print(arr / arr)                 # [1. 1. 1. ...]
9 print(arr ** 2)                  # [ 1  4  9 16 25 36 49 64 81]
10
11 # Matrix multiplication (inner product for 1D)
12 print(arr @ arr)                # 285 (= 1*1 + 2*2 + ... + 9*9)
13 # Same as: np.sum(arr * arr)
14
15 # Universal functions / 通用数学函数
16 print(np.log(arr))
17 print(np.log2(arr))
18 print(np.exp(arr))
19 print(np.sin(arr))
```

【Example / 实战案例】 Aggregation with axis / 聚合与轴参数

```
1 two_dim_arr = np.random.randint(0, high=20, size=(3, 4))
2 print(two_dim_arr)              # 3x4 random array
3
4 # Aggregation over whole array / 对整个数组聚合
5 print(np.min(two_dim_arr))      # Minimum of ALL elements
6 print(np.max(two_dim_arr))      # Maximum of ALL elements
7 print(np.sum(two_dim_arr))      # Sum of ALL elements
8
9 # axis=0: Operation applied across rows (column-wise)
10 # 纵向聚合: 按列计算, 结果 shape 为 (4,)
11 print(np.min(two_dim_arr, axis=0)) # Min of each column
12
13 # axis=1: Operation applied across columns (row-wise)
14 # 横向聚合: 按行计算, 结果 shape 为 (3,)
15 print(np.average(two_dim_arr, axis=1)) # Avg of each row
```

【Example / 实战案例】 concatenate, append, stack / 数组合并三剑客

```
1 a = np.arange(10)               # [0 1 2 3 4 5 6 7 8 9]
2 b = np.arange(10)[::-1]         # [9 8 7 6 5 4 3 2 1 0]
3
4 # concatenate: joins a sequence of arrays
5 print(np.concatenate((a, b)))
6 # [0 1 2 3 4 5 6 7 8 9 9 8 7 6 5 4 3 2 1 0]
7
8 # append: uses concatenate internally
```

```
9 print(np.append(a, b))
10
11 # stack: stacks along a NEW axis (creates new dimension)
12 print(np.stack((np.arange(10), np.arange(10))))
13 # [[0 1 2 3 4 5 6 7 8 9]
14 #   [0 1 2 3 4 5 6 7 8 9]]
```

【Example / 实战案例】Masking Examples / 掩码过滤示例

```
1 arr = np.arange(1, 6)                # [1 2 3 4 5]
2
3 # Create a boolean mask
4 mask = np.array([True, False, True, False, True])
5 print(arr[mask])                     # [1 3 5] (only where mask is True)
6
7 # Generate mask from condition
8 print(arr < 5)                       # [ True  True  True  True False]
9 print(arr[arr < 3])                  # [1 2] (filtered by condition)
```

【Example / 实战案例】2D Array Indexing / 二维数组索引

The syntax works as [row, column], or more precisely [row_start:row_end, col_start:col_end].

```
1 large_two_dim_arr = np.arange(81).reshape((9, 9))
2
3 # Select entire column / 选择整列
4 print(large_two_dim_arr[:, 2])       # All rows, column 2
5
6 # Select specific row and column range
7 print(large_two_dim_arr[1, 2:7])     # Row 1, columns 2 through 6
8
9 # Standard slicing with step / 带步长的切片
10 print(large_two_dim_arr[:, 2:7:2])  # All rows, columns 2, 4, 6
```

【Example / 实战案例】Minimal Class / 最简类定义

```
1 class Employee:
2     pass                               # Placeholder — does nothing, but avoids SyntaxError
```

【Example / 实战案例】Class Definition with Attributes / 完整类定义

```
1 class Employee:
2     # Class Attribute / 类属性 — shared by all instances
3     company_name = "Emlyon"
4
5     def __init__(self, name, age):
```

```
6     # Instance Attributes / 实例属性 — unique to each instance
7     self.name = name
8     self.age = age
```

【Example / 实战案例】Creating Objects / 创建对象

```
1 emp1 = Employee("David", 23)
2 emp2 = Employee("Alice", 35)
3
4 print(emp1)                # <__main__.Employee object at 0x00aeff70>
5 # The hex number is the memory address — different on each computer!
6
7 print(emp1 == emp2)        # False (different objects in memory)
8 print(type(emp1))          # <class '__main__.Employee'>
9
10 # Access attributes with dot notation / 用点号访问属性
11 print(emp1.name)           # 'David'
12 print(emp2.age)           # 35
13
14 # Class attributes are accessed the same way
15 print(emp1.company_name)   # 'Emlyon'
16 print(emp2.company_name)   # 'Emlyon' (same for all)
```

【Example / 实战案例】Instance Methods Example / 实例方法案例

```
1 class Employee:
2     company_name = "Emlyon"
3
4     def __init__(self, name, age):
5         self.name = name
6         self.age = age
7
8     # Instance method / 实例方法
9     def description(self):
10         return f"{self.name} is {self.age} years old"
11
12     # Another instance method / 另一个实例方法
13     def update_age(self, new_age):
14         self.age = new_age
15
16 # Usage
17 emp1 = Employee("David", 23)
18 print(emp1.description())    # 'David is 23 years old'
19 emp1.update_age(24)
20 print(emp1.description())    # 'David is 24 years old'
```


【Example / 实战案例】 Customizing print() Output / 自定义 print() 输出

```
1 class Employee:
2     company_name = "Emlyon"
3
4     def __init__(self, name, age):
5         self.name = name
6         self.age = age
7
8     def __str__(self):
9         return f"{self.name} is {self.age} years old"
10
11 emp1 = Employee("David", 23)
12 print(emp1)                                # 'David is 23 years old' — nice!
```

【Example / 实战案例】 Full Inheritance Example / 继承完整案例

```
1 # Parent class / 父类
2 class Employee:
3     company_name = "Emlyon"
4
5     def __init__(self, name, age):
6         self.name = name
7         self.age = age
8
9     def __str__(self):
10        return f"{self.name} is {self.age} years old"
11
12 # Child class 1: HR Employee / 子类 1
13 class HREmployee(Employee):
14     def __init__(self, name, age, wh):
15         super().__init__(name, age)      # Call parent's __init__
16         self.working_hours = wh          # New attribute / 新属性
17
18     def update_working_hours(self, new_hour): # New method / 新方法
19         self.working_hours = new_hour
20
21     def __str__(self):                     # Override __str__ / 重写
22         return f"{self.name} is {self.age} years old and works in HR and has {self.
23             working_hours} working hours!"
24
25 # Child class 2: Management Employee / 子类 2
26 class ManagementEmployee(Employee):
27     def __init__(self, name, age):
28         super().__init__(name, age)
29         self.emp_list = []               # New attribute / 新属性
30
31     def add_employee(self, emp_name):      # New method / 新方法
32         self.emp_list.append(emp_name)
33
34     def give_emp_list(self):               # New method / 新方法
35         for i in range(len(self.emp_list)):
```

```
35         print(f"{i} : {self.emp_list[i]}")
```

【Example / 实战案例】Using the Child Classes / 使用子类

```
1  # HR Employee / HR 员工
2  hr_emp_1 = HREmployee("David", 23, 8)
3  print(hr_emp_1)                # Uses overridden __str__
4  hr_emp_1.update_working_hours(10) # Child's own method
5  print(hr_emp_1.working_hours)    # 10
6
7  # Management Employee / 管理员工
8  management_emp_1 = ManagementEmployee("Alice", 35)
9  print(management_emp_1)         # Uses parent's __str__
10 management_emp_1.add_employee("John")
11 management_emp_1.add_employee("Maria")
12 management_emp_1.give_emp_list()
13 # Output:
14 # 0 : John
15 # 1 : Maria
```

【Example / 实战案例】@property Example / @property 案例

```
1  class MyClass:
2      def __init__(self):
3          self._hidden = 42          # Prefix '_' suggests internal use
4
5      @property
6      def hidden(self):              # Getter — can access like an attribute
7          return self._hidden
8
9      def get_hidden(self):          # Ordinary getter — requires parentheses
10         return self._hidden
11
12  my_object = MyClass()
13
14  # @property: access like an attribute (no parentheses!)
15  print(my_object.hidden)           # 42
16
17  # Direct assignment to @property is BLOCKED
18  # my_object.hidden = 0             # AttributeError! Read-only!
19
20  # BUT: _hidden can still be accessed directly if you really want
21  my_object._hidden = 0             # Works, but you shouldn't do it
22  print(my_object._hidden)          # 0
23
24  # Ordinary getter requires parentheses
25  print(my_object.get_hidden())     # 0
```

【Example / 实战案例】@property.setter Example / @property.setter 案例（带验证的写入）

通过 @xxx.setter 可以在赋值时自动执行验证逻辑——阻止非法值写入：

```
1 class Student:
2     def __init__(self, name, score):
3         self.name = name
4         self._score = score                # Internal storage
5
6     @property
7     def score(self):                       # Getter — read like attribute
8         return self._score
9
10    @score.setter
11    def score(self, value):                # Setter — validate before writing
12        if value < 0 or value > 100:
13            raise ValueError("Score must be between 0 and 100!")
14        self._score = value
15
16 s = Student("Alice", 85)
17 print(s.score)                            # 85 — reads like attribute (no parentheses)
18
19 s.score = 92                              # Valid — setter silently accepts
20 print(s.score)                            # 92
21
22 # s.score = 150                            # ValueError: Score must be between 0 and 100!
23 # s.score = -10                           # ValueError — blocked by setter validation!
24
25 # Key advantage: the validation is REUSABLE — every assignment
26 # to .score automatically goes through the setter's check.
```

【Example / 实战案例】Polymorphism via Inheritance / 通过继承实现多态

子类重写（override）父类方法，同一个方法名在不同子类中表现不同：

```
1 # Parent class / 父类
2 class Animal:
3     def speak(self):
4         return "Some sound"
5
6 # Child classes override speak() / 子类各自重写 speak()
7 class Dog(Animal):
8     def speak(self):
9         return "Woof! 汪汪! "
10
11 class Cat(Animal):
12     def speak(self):
13         return "Meow! 喵喵! "
14
15 class Duck(Animal):
16     def speak(self):
17         return "Quack! 嘎嘎! "
18
```

```
19 # Polymorphism in action: same method, different behavior!
20 animals = [Dog(), Cat(), Duck(), Dog()]
21 for a in animals:
22     print(a.speak())
23 # Output:
24 # Woof! 汪汪!
25 # Meow! 喵喵!
26 # Quack! 嘎嘎!
27 # Woof! 汪汪!
```

关键：调用 `a.speak()` 时，Python 自动根据对象的**实际类型**选择正确的 `speak()` 实现——这就是多态的核心。

【Example / 实战案例】Duck Typing Polymorphism / 鸭子类型的多态（Python 特色）

Python 不需要继承自同一个父类，只要对象有同名方法就能互换：

```
1 # These classes don't inherit from Animal at all!
2 class Car:
3     def start(self):
4         return "Engine roaring! Vroom vroom!"
5
6 class Computer:
7     def start(self):
8         return "Booting up... Beep!"
9
10 class Morning:
11     def start(self):
12         return "Time to wake up! 起床啦! "
13
14 # All can be used interchangeably — duck typing!
15 things = [Car(), Computer(), Morning()]
16 for t in things:
17     print(t.start())
18 # Output:
19 # Engine roaring! Vroom vroom!
20 # Booting up... Beep!
21 # Time to wake up! 起床啦!
```

这就是 Python 的”鸭子类型”哲学：不关心你是什么类型，只关心你有没有我需要的方法。

【Example / 实战案例】Destructor Example / 析构函数示例

```
1 class FileHandler:
2     def __init__(self, filename):
3         self.filename = filename
4         self.file = open(filename, 'w')
5         print(f"File '{filename}' opened.")
6
7     def write_data(self, data):
8         self.file.write(data)
9
```

```
10     def __del__(self):
11         self.file.close()
12         print(f"File '{self.filename}' closed (cleanup done).")
13
14 # Usage / 使用
15 handler = FileHandler("test.txt")
16 handler.write_data("Hello, world!")
17
18 # When 'handler' goes out of scope or the program ends,
19 # __del__() is called automatically to close the file.
```

最佳实践：析构函数适合做“保险措施”——即使你忘了手动清理，对象销毁时也会自动调用 `__del__()`。但不要完全依赖它，关键资源还是用 `with` 或显式 `.close()` 来管理。

【Example / 实战案例】Checking File Attributes / 查看文件属性

```
1 my_file = open(file='sample.txt', mode='w')
2
3 print("Name of the file: ", my_file.name)      # sample.txt
4 print("Closed or not: ", my_file.closed)       # False
5 print("Opening mode: ", my_file.mode)         # w
```

【Example / 实战案例】write() Method / write() 方法

The `write()` method writes any string to an open file. **IMPORTANT:** It does **NOT** add a newline character (`\n`) automatically—you must add it manually!

`write()` 不会自动添加换行符，必须手动加 `\n`!

```
1 my_file = open(file='sample.txt', mode='w')
2 my_file.write('Hello file!')
3 my_file.write('\n')                # Manually add newline
4 my_file.write('This is me!')
5 my_file.close()                    # Always close to flush data!
```

【Example / 实战案例】The with Statement (Best Practice) / with 语句（最佳实践）

Recommended: Use `with open(...) as f:` —it automatically closes the file, even if an exception occurs. This avoids the common mistake of forgetting `close()`.

推荐：使用 `with` 语句自动关闭文件，即使发生异常也不会泄漏资源。

```
1 # OLD WAY — must remember to close (easy to forget!)
2 my_file = open('sample.txt', 'w')
3 my_file.write('Hello!\n')
4 my_file.close()                # Don't forget this!
5
6 # BETTER WAY — with statement auto-closes!
7 with open('sample.txt', 'w') as f:
8     f.write('Hello!\n')        # No need to call close()
9     f.write('World!\n')
```

```
10 # File is automatically closed here — even if an error occurred!
11
12 # Reading with `with` + iterating
13 with open('sample.txt', 'r') as f:
14     for line in f:                # Memory-efficient iteration
15         print(line.strip())       # Strip removes trailing \n
16
17 # `with` is the Pythonic way — it works with ALL file modes
18 # and guarantees cleanup. Use it in every exam answer!
```

【Example / 实战案例】Reading Methods Comparison / 四种读取方式对比

```
1 # Method 1: .read() — entire file as one string
2 my_file = open(file='sample.txt', mode='r')
3 file_output = my_file.read()
4 my_file.close()
5 print(file_output)
6
7 # Method 2: .readline() — just one line
8 my_file = open(file='sample.txt', mode='r')
9 file_output = my_file.readline()    # Reads first line
10 my_file.close()
11
12 # Method 3: .readlines() — list of lines
13 my_file = open(file='sample.txt', mode='r')
14 file_output = my_file.readlines()
15 my_file.close()
16 print(file_output)                # ['Hello file!\n', 'This is me!\n']
17
18 # Method 4: for line in file — BEST for large files!
19 my_file = open(file='sample.txt', mode='r')
20 for line in my_file:               # Most memory-efficient
21     print(line)
22 my_file.close()
```

【Example / 实战案例】Using csv.reader() and DictReader / CSV 读取器

```
1 import csv
2
3 # Method 1: csv.reader — returns lists
4 csv_file = open("sample.csv", "r")
5 reader = csv.reader(csv_file, skipinitialspace=True)
6 for row in reader:
7     print(row)                    # ['1', 'Rose', '30', 'Grenoble']
8 csv_file.close()
9
10 # Method 2: csv.DictReader — returns dicts (RECOMMENDED!)
11 csv_file = open("sample.csv", "r")
12 reader = csv.DictReader(csv_file)
13 for row in reader:
```

```
14     print(row)
15     # {'id': '1', 'name': 'Rose', 'age': '30', 'city': 'Grenoble'}
16 csv_file.close()
```

【Example / 实战案例】Using csv.writer() / CSV 写入器

```
1 # Write CSV — use "w" to overwrite, "a" to append
2 csv_file = open("sample.csv", "w")
3 writer = csv.writer(csv_file)
4 writer.writerow(['5', 'Rose', '30', 'Grenoble'])
5 writer.writerow(['6', 'Nona', '23', 'Toulouse'])
6 csv_file.close()
```

【Example / 实战案例】Common Error Examples / 常见错误类型

```
1 # SyntaxError: invalid name (starts with digit)
2 # 3_apples = ["apple", "apple", "apple"]
3
4 # SyntaxError: forgot to close brackets
5 # three_apples = ["apple", "apple", "apple"
6
7 # SyntaxError: forgot colon and wrong indentation
8 # for i in range(3)
9 #     print(three_apples[i])
10
11 # IndexError: index out of range
12 three_apples = ['A', 'B', 'C']
13 # print(three_apples[4])           # IndexError!
14
15 # TypeError: cannot concatenate str + int
16 # print("hello " + 6)             # TypeError!
17
18 # ValueError: invalid literal for float()
19 converting_data_types = "I am a string"
20 # print(float(converting_data_types)) # ValueError!
```

【Example / 实战案例】Specific Exception Handling / 精确捕获异常类型

永远避免裸 `except:`! 始终指定具体的异常类型:

```
1 # BAD — bare except catches EVERYTHING (even Ctrl+C!)
2 try:
3     result = 10 / 0
4 except:                               # Don't do this!
5     print("Something went wrong")
6
7 # GOOD — catch specific exceptions
8 try:
9     user_input = input("Enter a number: ")
```

```
10     result = 10 / float(user_input)
11 except ValueError:                                # Catches bad float conversion
12     print("That's not a valid number!")
13 except ZeroDivisionError:                          # Catches division by zero
14     print("Cannot divide by zero!")
```

【Example / 实战案例】 else, finally, and raise / else · finally · raise 完整结构

```
1 def safe_divide(a, b):
2     try:
3         result = a / b
4     except ZeroDivisionError:
5         print("Error: Division by zero!")
6         raise                                # Re-raise — let caller handle it
7     except TypeError as e:                    # Capture the exception object
8         print(f"Type error: {e}")
9         raise ValueError("Both arguments must be numbers") from e
10    else:
11        print(f"Success! Result = {result}") # Only runs if no exception
12        return result
13    finally:
14        print("Cleanup: this ALWAYS runs.") # Runs no matter what!
15
16 # Test cases / 测试
17 safe_divide(10, 2)      # Success → result=5.0, cleanup runs
18 # safe_divide(10, 0)    # ZeroDivisionError → cleanup STILL runs!
19 # safe_divide(10, "x")  # TypeError → chained to ValueError
20
21 # Key points:
22 # - else: runs ONLY on success (avoids catching its own exceptions)
23 # - finally: ALWAYS runs — even after return/raise/exception
24 # - raise: re-throws the same exception to the caller
25 # - raise X from Y: chains exceptions for better debugging
```

【Example / 实战案例】 Debugging Workflow Demo / 调试流程演示

```
1 # Problem: function returns unexpected result
2 def calculate_average(grades):
3     total = 0
4     for g in grades:
5         total += g
6     return total / len(grades)
7
8 grades = [85, 92, 78]
9 result = calculate_average(grades)
10 print(f"Result: {result}")      # 85.0 — correct!
11
12 # But what if we have an empty list?
13 # calculate_average([])          # ZeroDivisionError — crash!
14
```



```
15 # Debugging approach:
16 # 1. Add print to check input
17 def calculate_average_v2(grades):
18     print(f"DEBUG: grades = {grades}, type = {type(grades)}") # Check input
19     if len(grades) == 0:
20         return 0 # Guard clause / 防御性检查
21     total = 0
22     for g in grades:
23         total += g
24     result = total / len(grades)
25     print(f"DEBUG: total={total}, count={len(grades)}, avg={result}")
26     return result
27
28 print(calculate_average_v2([])) # 0 — safe!
29 print(calculate_average_v2([1, 2, 3, 4, 5])) # 3.0 — verified!
```

【Example / 实战案例】 Good vs Bad Formatting / 好代码 vs 坏代码

```
1 # Bad / 坏代码
2 def my_function ( a,b,c ) :
3     some_dict={"a" : "b", 4:5}
4     another_function(a+b+c, n = 42)
5
6 # Good / 好代码
7 def my_function(a, b, c):
8     some_dict = {"a": "b", 4: 5}
9     another_function(a + b + c, n=42)
```

【Example / 实战案例】 Line Continuation / 行续接

```
1 # Explicit continuation with backslash
2 total_income = gross_income \
3                 - taxes \
4                 - student_loan_interest
```

【Example / 实战案例】 The First Plot / 第一个绘图

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 my_list = [10, 20, 25, 35]
5 plt.plot(my_list) # Simple line plot
6 # In Jupyter: the plot displays automatically
7 # In scripts: add plt.show() to display
```

【Example / 实战案例】 Using plt.subplots() / 显式创建 Figure 和 Axes

To manage multiple plots, it makes sense to explicitly create `figure` and `axes` objects.

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 # Create a single plot — returns (figure, axes)
5 fig, ax = plt.subplots(nrows=1, ncols=1)
6
7 # Create a grid of subplots — 2 rows, 4 columns
8 fig, axes = plt.subplots(nrows=2, ncols=4)
9 # axes is now a 2x4 array of Axes objects
```

【Example / 实战案例】 Improving Appearance / 改善外观

```
1 # Without tight_layout — labels may overlap!
2 fig, axes = plt.subplots(nrows=2, ncols=4)
3 fig.tight_layout() # Fix overlapping
4
5 # With figsize — control the overall size
6 fig, axes = plt.subplots(nrows=2, ncols=4, figsize=(10, 4))
7 fig.tight_layout()
```

【Example / 实战案例】 Using Explicit Setters / 显式配置方法

Matplotlib's objects have lots of explicit setters — functions that start with `set_<something>`:

```
1 fig, ax = plt.subplots(nrows=1, ncols=1)
2
3 ax.set_xlim([0.5, 4.5]) # Set x-axis range
4 ax.set_ylim([-2, 8]) # Set y-axis range
5 ax.set_title("An Example Axes") # Set title
6 ax.set_ylabel("Y-Axis") # Set y-axis label
7 ax.set_xlabel("X-Axis") # Set x-axis label
8
9 fig # In Jupyter: display the figure
```

【Example / 实战案例】 Dynamically Colored Bars / 动态着色条形图

```
1 np.random.seed(1)
2 x = np.arange(5)
3 y = np.random.randn(5)
4
5 fig, ax = plt.subplots()
6 vertBars = ax.bar(x, y)
7
8 # Color negative bars red / 负值柱体标红
9 for bar, height in zip(vertBars, y):
10     if height < 0:
```

```
11     bar.set(color='red')
12
13 ax.axhline(y=0, color='black', linewidth=2) # Add zero line
```

【Example / 实战案例】Multidimensional Scatter / 多维散点图

```
1 n = 50
2 x1 = np.random.random(n)
3 x2 = np.random.random(n) * 50
4 x3 = np.random.random(n)
5 y = x1 + x2 + x3
6
7 fig, ax = plt.subplots()
8 sc = ax.scatter(x=x1, y=y, s=x2, c=x3, marker='o')
9 fig.colorbar(sc) # Add color scale bar
```

【Example / 实战案例】Using imshow() / imshow() 显示图像

```
1 fig, ax = plt.subplots()
2 img = plt.imread("images/cat.jpg")
3 ax.imshow(img)
4 ax.axis("off") # Hide axis ticks and labels
```

【Example / 实战案例】Color Specification Examples / 颜色指定示例

```
1 t = np.arange(0.0, 5.0, 0.2)
2 fig, ax = plt.subplots()
3
4 # Single letters
5 ax.plot(t, t, linewidth=5, color="r")
6
7 # Hex values
8 ax.plot(t, t**2, linewidth=5, color='#00ffff')
9
10 # RGB tuple (with alpha)
11 ax.plot(t, t**3, linewidth=5, color=(0, 0, 1, 0.5))
12
13 # Grayscale
14 ax.plot(t, -t, linewidth=5, color='0.5')
15 plt.show()
```

【Example / 实战案例】Marker and Linestyle Examples / 标记与线型示例

```
1 t = np.arange(0.0, 5.0, 0.2)
2 fig, ax = plt.subplots()
3
```

```
4 # With explicit arguments, you can set marker and linestyle separately
5 ax.plot(t, t, linestyle='-', linewidth=5)
6 ax.plot(t, t**2, linestyle='--', linewidth=5)
7 ax.plot(t, t**3, linestyle='-.', linewidth=5)
8 ax.plot(t, -t, linestyle=':', linewidth=5)
9 plt.show()
```

【Example / 实战案例】Colormap Comparison / 色图对比

```
1 example_x = np.random.normal(size=500)
2 example_y = np.random.normal(size=500)
3 example_z = example_x + example_y
4
5 plt.scatter(example_x, example_y, c=example_z, cmap="jet")
6 plt.show()
7
8 plt.scatter(example_x, example_y, c=example_z, cmap="viridis")
9 plt.show()
```

【Example / 实战案例】Mathtext Example / 数学公式示例

```
1 fig, ax = plt.subplots()
2 ax.scatter([1, 2, 3, 4], [4, 3, 2, 1])
3 ax.spines['top'].set(visible=False)      # Remove top border
4 ax.spines['right'].set(visible=False)    # Remove right border
5 ax.set_title(r'$\sigma_i = \frac{3}{5}$', fontsize=25)
6 plt.show()
```

【Example / 实战案例】Legend Example / 图例示例

```
1 fig, ax = plt.subplots()
2 ax.plot([1, 2, 3, 4], [10, 20, 25, 30], label='Paris')
3 ax.plot([1, 2, 3, 4], [30, 23, 13, 4], label='Lyon')
4 ax.set(ylabel='Temperature (deg C)', xlabel='Time', title='A tale of two cities')
5 ax.legend()                             # Automatically builds legend from labels
6 # ax.legend(loc='center')               # Use loc to position the legend
7 plt.show()
```

【Example / 实战案例】Manual Tick Configuration / 手动配置刻度

```
1 fig, ax = plt.subplots()
2 ax.plot([1, 2, 3, 4], [10, 20, 25, 35])
3
4 # Manually set ticks and tick labels on the x-axis
5 ax.xaxis.set(ticks=range(1, 5), ticklabels=[3, 100, -12, "foo"])
6
```

```
7 # Customize tick appearance
8 ax.tick_params(axis='y', direction='in', length=10)
9 plt.show()
```

【Example / 实战案例】 Tick Labels from Data / 从数据生成刻度标签

```
1 data = [('apples', 2), ('oranges', 3), ('peaches', 1)]
2 fruit, value = zip(*data)           # Unzip into two tuples
3
4 fig, ax = plt.subplots()
5 x = np.arange(len(fruit))
6 ax.bar(x, value, align='center', color='gray')
7 ax.set(xticks=x, xticklabels=fruit)
8 plt.show()
```

【Example / 实战案例】 Removing Spines / 移除边框

```
1 ax.spines['top'].set(visible=False)    # Remove top border
2 ax.spines['right'].set(visible=False)  # Remove right border
3 # Also available: 'bottom', 'left'
```

【Example / 实战案例】 Subplot Spacing / 子图间距调整

```
1 fig, axes = plt.subplots(2, 2, figsize=(9, 9))
2 fig.subplots_adjust(wspace=0.3, hspace=-0.7,
3                     left=0.125, right=0.8,
4                     top=0.7, bottom=0.2)
5 plt.show()
```

【Example / 实战案例】 Tight Layout for Multiple Subplots / 多子图的紧凑布局

```
1 fig, ((ax1, ax2), (ax3, ax4)) = plt.subplots(nrows=2, ncols=2)
2
3 # Without tight_layout — labels overlap
4 example_plot(ax1); example_plot(ax2)
5 example_plot(ax3); example_plot(ax4)
6 plt.show()
7
8 # With tight_layout — everything fits!
9 fig.tight_layout()
10 plt.show()
```

【Example / 实战案例】Creating DataFrame from Dict / 字典 → DataFrame

Dict keys become column labels; dict values become column data.

```
1 import pandas as pd
2
3 # Simple dict / 简单字典
4 student_dict = {'Name': ['Joe', 'Nat'], 'Age': [20, 21], 'Marks': [85.10, 77.80]}
5 student_df = pd.DataFrame(student_dict)
6 print(student_df)
7 #      Name  Age  Marks
8 # 0   Joe   20  85.10
9 # 1   Nat   21  77.80
10
11 # With custom indices / 自定义索引
12 df = pd.DataFrame({'A': [1, 2, 3], 'B': [True, True, False],
13                    'C': [0.496714, -0.138264, 0.647689]},
14                    index=['i', 'ii', 'iii'])
15 print(df)
```

【Example / 实战案例】Creating DataFrame from List / 列表 → DataFrame

By default, all list elements are added as a **single row**.

```
1 # Simple list — becomes one row / 列表变单行
2 fruits_list = ['Apple', 10, 'Orange', 55.50]
3 fruits_df = pd.DataFrame(fruits_list)
4 print(fruits_df)
5
6 # With custom column names and index / 自定义列名和索引
7 fruits_list = ['Apple', 'Banana', 'Orange', 'Mango']
8 fruits_df = pd.DataFrame(fruits_list, columns=['Fruits'],
9                          index=['Fruit1', 'Fruit2', 'Fruit3', 'Fruit4'])
10 print(fruits_df)
11
12 # Multi-dimensional list / 多维列表
13 fruits_list = [['Apple', 'Banana', 'Orange', 'Mango'], [120, 40, 80, 500]]
14 fruits_df = pd.DataFrame(fruits_list)
15 print(fruits_df)      # Each sublist becomes a row
16
17 # Transpose to swap rows/columns / 转置行列互换
18 fruits_df = pd.DataFrame(fruits_list).transpose()
19 print(fruits_df)
```

【Example / 实战案例】Reading CSV Files / 读取 CSV 文件

`pd.read_csv("file.csv")` is the most common way to load large datasets.

```
1 data1 = pd.read_csv("data/sample.csv")
2 print(type(data1))      # <class 'pandas.core.frame.DataFrame'>
3 print(data1.iloc[:, 2])  # Select 3rd column by position
```

【Example / 实战案例】Indexing Examples / 索引实战

```
1 df = pd.DataFrame({'A': [1, 2, 3], 'B': [True, True, False],
2                     'C': [0.496714, -0.138264, 0.647689]},
3                     index=['i', 'ii', 'iii'])
4
5 # Column Selection / 列选择
6 print(df['A'])                # Single column → returns Series
7 print(df[['A', 'C']])        # Multiple columns → returns DataFrame
8
9 # Row Selection with .loc (label-based)
10 print(df.loc[['i', 'iii']])  # Select rows 'i' and 'iii'
11 print(df.loc['i':'iii'])     # INCLUSIVE of 'iii'!
12
13 # Row Selection with .iloc (position-based)
14 print(df.iloc[[0, 1]])       # Select rows 0 and 1
15 print(df.iloc[:2])           # EXCLUSIVE of index 2
16
17 # Combined row + column selection / 行列同时选择
18 print(df.loc['ii', 'C'])     # Scalar value at row 'ii', column 'C'
19 print(df.loc['i':'ii', ['A', 'C']]) # Rows 'i' to 'ii', columns A and C
20
21 # Scalar access / 单值快速访问
22 print(df.at[0, 'Name'])      # Fast label-based single value
23 print(df.iat[0, 0])          # Fast position-based single value
```

【Example / 实战案例】Creating and Selecting Series / Series 的创建与选择

```
1 # Three ways to select a column as Series / 三种选取列的方式
2 df['A']                # __getitem__ style
3 df.loc[:, 'A']         # .loc style (all rows, column 'A')
4 df.A                   # Attribute lookup style
5
6 # Create a Series directly / 直接创建 Series
7 s = pd.Series([2, 4, 6, 8, 10])
8 print(s)
9 # 0    2
10 # 1    4
11 # 2    6
12 # 3    8
13 # 4   10
14 # dtype: int64
```

Series share many of the same methods as DataFrames.

【Example / 实战案例】Metadata and Statistics / 元数据与统计示例

```
1 data1 = pd.read_csv("data/sample.csv")
2 data1.info()           # Full metadata summary
3 data1.describe()       # Statistics for numerical columns only
4 print(data1.shape)     # (rows, columns)
```

【Example / 实战案例】head, tail, at, iat, get / 选择函数

```
1 student_dict = {'Name': ['Joe', 'Nat', 'Harry'], 'Age': [20, 21, 19],
2                 'Marks': [85.10, 77.80, 91.54]}
3 student_df = pd.DataFrame(student_dict)
4
5 # Top N rows / 前 N 行
6 student_df.head(2)
7
8 # Bottom N rows / 后 N 行
9 student_df.tail(2)
10
11 # Single value by label / 按标签获取单值
12 student_df.at[0, 'Name']          # 'Joe'
13
14 # Single value by position / 按位置获取单值
15 student_df.iat[0, 0]              # 'Joe'
16
17 # Get column / 获取列
18 student_df.get('Name')            # Returns Series
```

【Example / 实战案例】df.insert() / 插入列

```
df.insert(loc=col_position, column=new_col_name, value=default_value)
```

```
1 student_dict = {'Name': ['Joe', 'Nat', 'Harry'], 'Age': [20, 21, 19],
2                 'Marks': [85.10, 77.80, 91.54]}
3 student_df = pd.DataFrame(student_dict)
4
5 # Insert new column 'Class' at position 2 with default value 'A'
6 student_df.insert(loc=2, column="Class", value='A')
7 print(student_df)
```

【Example / 实战案例】df.drop() / 删除列

```
df.drop(columns=[col1, col2, ...])
```

```
1 # Delete the 'Age' column
2 student_df = student_df.drop(columns='Age')
3 print(student_df)
```

【Example / 实战案例】Conditional Replacement / 条件替换示例

```
1 student_dict = {'Name': ['Joe', 'Nat', 'Harry'], 'Age': [20, 21, 19],
2                 'Marks': [85.10, 77.80, 91.54]}
3 student_df = pd.DataFrame(student_dict)
4
```



```
5 # Replace marks < 80 with 0
6 filter = student_df['Marks'] > 80
7 student_df['Marks'].where(filter, other=0, inplace=True)
8 print(student_df)
9 # Joe: 85.10 stays (True), Nat: 77.80 → 0 (False), Harry: 91.54 stays (True)
```

【Example / 实战案例】.filter() Example / .filter() 示例

```
1 student_df = pd.DataFrame({'Name': ['Joe', 'Nat', 'Harry'],
2                             'Age': [20, 21, 19],
3                             'Marks': [85.10, 77.80, 91.54]})
4
5 # Only include columns whose label starts with 'N'
6 student_df = student_df.filter(like='N', axis='columns')
7 print(student_df) # Only 'Name' column remains
```

【Example / 实战案例】df.rename() / 列重命名

```
df.rename(columns={'old_name': 'new_name'})

1 student_df = student_df.rename(columns={'Marks': 'Percentage'})
2 print(student_df)
```

【Example / 实战案例】df.join() / 横向拼接

df1.join(df2) joins DataFrames by index.

```
1 student_df = pd.DataFrame({'Name': ['Joe', 'Nat'], 'Age': [20, 21]})
2 marks_df = pd.DataFrame({'Marks': [85.10, 77.80]})
3 joined_df = student_df.join(marks_df)
4 print(joined_df)
```

【Example / 实战案例】Merge Examples / 合并示例

```
1 df1 = pd.DataFrame({'Name': ['Joe', 'Nat', 'Davis'], 'Age': [20, 21, 23]})
2 df2 = pd.DataFrame({'Name': ['Joe', 'Nat', 'Alice'], 'Marks': [85.10, 77.80, 90.00]})
3
4 print(pd.merge(df1, df2, how='inner', on='Name')) # Joe, Nat only
5 print(pd.merge(df1, df2, how='left', on='Name')) # Joe, Nat, Davis
6 print(pd.merge(df1, df2, how='right', on='Name')) # Joe, Nat, Alice
7 print(pd.merge(df1, df2, how='outer', on='Name')) # Joe, Nat, Davis, Alice
```

【Example / 实战案例】GroupBy Example / 分组聚合示例

```
1 student_df = pd.DataFrame({'Name': ['Joe', 'Nat', 'Harry'],
2                             'Class': ['A', 'B', 'A'],
```

```
3             'Marks': [85.10, 77.80, 91.54]})
4
5 # Average marks per class / 每个班级的平均分
6 result = student_df.groupby('Class').mean()
7 print(result)
8 #           Marks
9 # Class
10 # A           88.32
11 # B           77.80
12
13 # Reset index to make 'Class' a column again
14 result = result.reset_index()
```

【Example / 实战案例】df.sort_values() / 排序

```
df.sort_values(by=['col1', 'col2'], ascending=[True, False])

1 student_df = pd.DataFrame({'Name': ['Joe', 'Nat', 'Harry'],
2                             'Age': [20, 21, 19],
3                             'Marks': [85.10, 77.80, 91.54]})
4
5 # Sort by 'Marks' ascending
6 student_df = student_df.sort_values(by=['Marks'])
7 # Sort by 'Marks' descending
8 student_df = student_df.sort_values(by=['Marks'], ascending=False)
```

【Example / 实战案例】df.iterrows() / 逐行遍历

Returns (index, row) pairs for each row.

```
1 student_df = pd.DataFrame({'Name': ['Joe', 'Nat'], 'Age': [20, 21], 'Marks': [85, 77]})
2
3 for index, row in student_df.iterrows():
4     print(index, row)
5 # 0 Name      Joe
6 #   Age       20
7 #   Marks     85
8 #   Name: 0, dtype: object
```

【Example / 实战案例】df.to_dict() / 转回字典

```
1 dict_result = student_df.to_dict()
2 print(dict_result)
3 # {'Name': {0: 'Joe', 1: 'Nat'}, 'Age': {0: 20, 1: 21}, ...}
```

【Example / 实战案例】 Initial Setup / 初始化配置

```
1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 # Pandas display options / 显示选项
6 pd.set_option('display.max_columns', 15)
7 pd.set_option('display.max_rows', 15)
8 pd.set_option('display.float_format', lambda x: '%.3f' % x)
```

【Example / 实战案例】 Reading the house.csv Dataset / 读取房屋数据集

```
1 data = pd.read_csv('data/house.csv')
2 print(data.shape)      # (rows, columns) — check dataset size
3 data.head(5)           # View first 5 rows
4 data.tail(5)           # View last 5 rows
```

【Example / 实战案例】 Dropping Columns / 删除列

```
1 print(data.columns)    # See all column names
2
3 # Drop columns that won't be used
4 data = data.drop(columns=["country", "postcode"])
5 print(data.shape)      # Reduced column count
6 data.head(5)
```

【Example / 实战案例】 NaN Handling Example / 缺失值处理示例

```
1 # Check for NaNs / 检查缺失值
2 print(data.isna().sum())
3 print(data.isnull().sum())      # Same as isna()
4
5 # Remove rows with any NaN / 删除含 NaN 的行
6 data.dropna(inplace=True)
7 print(data.shape)              # Fewer rows after dropping
8
9 # Alternative: fill NaNs with mean / 替代方案: 用均值填充
10 # data['column'].fillna(data['column'].mean(), inplace=True)
```

【Example / 实战案例】 Renaming for Convenience / 简化列名

Long column names are hard to type repeatedly. Rename them for convenience.

```
1 data.rename(columns={
2     'rentEstimate_currentPrice': 'rentPrice',
3     'saleEstimate_currentPrice': 'salePrice'
```

```
4 }, inplace=True)
5 data.head(5)
```

【Example / 实战案例】Type Conversion Example / 类型转换示例

```
1 print(data.dtypes)                # Check current types
2
3 # Convert 'propertyType' from object to category
4 data['propertyType'] = data.propertyType.astype('category')
5 data.head(5)
```

【Example / 实战案例】Boolean Flag Column / 布尔标记列

```
1 # Create a boolean column 'above_mean'
2 data['above_mean'] = data.salePrice > data.salePrice.mean()
3 data.head(5)
```

【Example / 实战案例】Using assign() / 使用 assign() 方法

assign() also creates new columns using lambda:

```
1 data.assign(above_mean=lambda x: x.salePrice > x.salePrice.mean())
2 data.head(5)
```

【Example / 实战案例】Multi-Condition Filtering / 多条件过滤

Use & (and), | (or) between conditions. Each condition must be wrapped in **parentheses**!

```
1 # Filter: flat type, bedrooms >= 2, bathrooms <= 2
2 filtered = data.loc[(data['propertyType'] == 'Purpose Built Flat') &
3                     (data['bedrooms'] >= 2.0) &
4                     (data['bathrooms'] <= 2.0)]
5 print(filtered)
```

【Example / 实战案例】Modifying Values Based on Condition / 条件修改值

```
1 # Check value distribution / 查看值分布
2 print(data['above_mean'].value_counts())
3
4 # Swap True/False labels / 交换标签
5 data.loc[data['above_mean'] == False, 'above_mean'] = 'yes'
6 data.loc[data['above_mean'] == True, 'above_mean'] = 'no'
7 data.head(5)
```

【Example / 实战案例】Multi-Column Sort / 多列排序

```
1 # Sort salePrice ascending, bathrooms descending
2 data.sort_values(['salePrice', 'bathrooms'], ascending=[True, False])
3 # Equivalent: ascending=[1, 0] where 1=ascending, 0=descending
```

【Example / 实战案例】Price Binning Example / 价格分箱示例

```
1 # Divide salePrice into 3 equal-width bins
2 data["salePriceBins"] = pd.cut(data["salePrice"], bins=3,
3                               labels=['low', 'medium', 'high'])
4 data.head(5)
5 print(data["salePriceBins"].value_counts()) # Count per bin
```

【Example / 实战案例】LabelEncoder Example / 标签编码

```
1 from sklearn.preprocessing import LabelEncoder
2
3 # Check current values / 查看当前值
4 print(data.tenure.value_counts())
5
6 # Encode: Freehold → 0, Leasehold → 1
7 tenure_encoder = LabelEncoder()
8 data['tenure'] = tenure_encoder.fit_transform(data['tenure'])
9 print(data.head(5))
10 print(data.tenure.value_counts())
11
12 # See the mapping / 查看映射关系
13 print({k: v for k, v in enumerate(list(tenure_encoder.classes_))})
```

【Example / 实战案例】One Hot Encoding Example / 独热编码

```
1 # Create dummy columns for 'tenure'
2 data_encoder = pd.get_dummies(data, columns=["tenure"])
3 print(data_encoder.shape) # Now has more columns
4 data_encoder.head()
```

【Example / 实战案例】Exploring by Bathrooms / 按浴室数量分析

```
1 print(data["bathrooms"].unique()) # All unique bathroom counts
2 print(data["bathrooms"].value_counts()) # Frequency of each count
3
4 print(data.groupby("bathrooms").size()) # Same as value_counts
5 print(data.groupby("bathrooms")['salePrice'].mean()) # Avg price per bathroom count
```

【Example / 实战案例】Pivot Table Example / 透视表示例

```
1 # Mean salePrice by bathrooms AND price bins
2 data.pivot_table(index='bathrooms',
3                   columns='salePriceBins',
4                   values='salePrice',
5                   aggfunc='mean')
```

This creates a 2D table: rows = bathroom count, columns = price bin, values = mean sale price.

【Example / 实战案例】Crosstab Example / 交叉表示例

```
1 pd.crosstab(index=data["bathrooms"], columns=data["salePriceBins"])
```

This shows: for each bathroom count, how many properties fall into each price bin.

【Example / 实战案例】General Statistics / 整体统计

```
1 data.describe()
2 # Returns count, mean, std, min, 25%, 50%, 75%, max for all numerical columns
```

【Example / 实战案例】Line Plot with Pandas / Pandas 折线图

`kind='line'` 是默认类型，适合展示随时间或顺序变化的趋势。DataFrame 的每一列自动变成一条线。

```
1 # 单列折线图: salePrice 随 rentPrice 的变化趋势
2 data.loc[100:110].plot(x="rentPrice", y="salePrice",
3                        title='salePrice vs rentPrice',
4                        ylabel='salePrice', alpha=0.8)
5 plt.show()
6
7 # 多列折线图: 每列一条线 (自动分色 + 图例)
8 data.loc[100:110].plot(y=["salePrice", "rentPrice"],
9                        title='Price Comparison')
10 plt.show()
11
12 # Series 也可直接 .plot()
13 data['salePrice'].plot(title='Sale Price Trend')
14 plt.show()
```

【Example / 实战案例】Vertical Bar Plot (kind='bar') / 垂直柱状图

柱状图适合比较分类数据的数值大小。

```
1 # 基本柱状图: 按 bathrooms 分组统计 salePrice 均值
2 data.groupby('bathrooms')['salePrice'].mean().plot(
3     kind='bar', title='Avg Sale Price by Bathrooms',
4     xlabel='Bathrooms', ylabel='Avg Price', rot=0)
5 plt.show()
```

```
6
7 # 从透视表绘制分组柱状图
8 plot_data = data.pivot_table(index='bathrooms', columns='salePriceBins',
9                               values='salePrice', aggfunc='mean')
10 ax = plot_data.plot(kind='bar', rot=0, xlabel='', ylabel='Price Mean',
11                    fontsize=8, figsize=(12, 1.5),
12                    title='Bathrooms | Price Bins')
13 ax.legend(title='', loc='center', bbox_to_anchor=(0.5, -0.3),
14           ncol=3, frameon=False)
15 plt.show()
```

【Example / 实战案例】Horizontal Bar Plot (kind='barh') / 水平柱状图

barh 特别适合类别名称较长的场景，避免标签重叠。

```
1 # 水平柱状图：各类别销售均值
2 data.groupby('propertyType')['salePrice'].mean().sort_values().plot(
3     kind='barh', title='Avg Sale Price by Property Type',
4     xlabel='Avg Price', color='steelblue')
5 plt.show()
```

【Example / 实战案例】Histogram (kind='hist') / 直方图

直方图展示数值数据的分布形态——集中趋势、离散程度、偏态。bins 参数控制柱子数量。

```
1 # 单列直方图：查看 salePrice 分布
2 data['salePrice'].plot(kind='hist', bins=30, title='Sale Price Distribution',
3                        edgecolor='black', alpha=0.7)
4 plt.show()
5
6 # 多列直方图叠加：对比不同列的分布
7 data[['salePrice', 'rentPrice']].plot(kind='hist', bins=30, alpha=0.5,
8                                       title='Price Distributions Comparison')
9 plt.show()
10
11 # 快速方法：df.hist() 为每列生成独立直方图
12 data[['salePrice', 'rentPrice', 'bedrooms']].hist(bins=20, figsize=(10, 6))
13 plt.tight_layout()
14 plt.show()
```

【Example / 实战案例】Box Plot (kind='box') / 箱线图

箱线图显示数据的五数概括：最小值、Q1（25% 分位）、中位数、Q3（75% 分位）、最大值，以及离群点。是发现异常值的首选工具。

```
1 # 单列箱线图
2 data['salePrice'].plot(kind='box', title='Sale Price Box Plot')
3 plt.show()
4
5 # 按分类分组绘制箱线图：比较不同 propertyType 的价格分布
```

```
6 data.boxplot(column='salePrice', by='propertyType', rot=45,
7             figsize=(8, 5))
8 plt.title('Sale Price by Property Type')
9 plt.suptitle('') # 移除 Pandas 自动添加的 suptitle
10 plt.show()
11
12 # 多列箱线图
13 data[['salePrice', 'rentPrice']].plot(kind='box',
14                                     title='Price Distributions')
15 plt.show()
```

【Example / 实战案例】Scatter Plot (kind='scatter') / 散点图

散点图展示两个数值变量之间的关系（相关性、聚类、离群点）。

```
1 # 基本散点图: salePrice vs rentPrice
2 data.plot(kind='scatter', x='rentPrice', y='salePrice',
3          title='salePrice vs rentPrice', alpha=0.5, s=10)
4 plt.show()
5
6 # 带颜色映射的散点图: 用第三维变量着色
7 data.plot(kind='scatter', x='rentPrice', y='salePrice',
8          c='bedrooms', colormap='viridis', alpha=0.6,
9          title='Price Scatter (color = bedrooms)', sharex=False)
10 plt.show()
11
12 # 矩阵散点图: pd.plotting.scatter_matrix 生成所有数值列的配对散点图
13 pd.plotting.scatter_matrix(data[['salePrice', 'rentPrice', 'bedrooms']],
14                          figsize=(10, 10), diagonal='hist', alpha=0.5)
15 plt.show()
```

【Example / 实战案例】Area Plot (kind='area') / 面积图

面积图是折线图的填充版本，适合展示组成部分随时间的变化。`stacked=True`（默认）堆叠显示总量构成。

```
1 # 堆叠面积图
2 data.groupby('bedrooms')['salePrice'].mean().plot(
3     kind='area', title='Avg Sale Price by Bedrooms (Stacked)',
4     alpha=0.4, figsize=(10, 5))
5 plt.show()
6
7 # 非堆叠面积图 (透明叠加)
8 data.groupby('bedrooms')['salePrice'].mean().plot(
9     kind='area', stacked=False, alpha=0.3,
10    title='Avg Sale Price by Bedrooms (Unstacked)')
11 plt.show()
```


【Example / 实战案例】Pie Chart (kind='pie') / 饼图

饼图展示各部分占整体的比例。适合类别数量较少（6）的场景。类别过多时考虑用柱状图替代。

```
1 # 饼图：各 propertyType 占比
2 type_counts = data['propertyType'].value_counts()
3 type_counts.plot(kind='pie', autopct='%1.1f%%',
4                  title='Property Type Distribution',
5                  ylabel='', legend=False, figsize=(6, 6))
6 plt.show()
7
8 # 饼图 + 自定义颜色 + 突出显示
9 colors = ['#ff9999', '#66b3ff', '#99ff99', '#ffcc99']
10 type_counts.plot(kind='pie', autopct='%1.1f%%', colors=colors,
11                  explode=(0, 0.05, 0, 0), # 突出第二块
12                  title='Property Types', ylabel='')
13 plt.show()
```

【Example / 实战案例】KDE Density Plot (kind='kde' / kind='density') / 核密度估计图

密度图是直方图的平滑版本，更适合展示数据分布的形状（多峰、偏态）。

```
1 # 单列密度图
2 data['salePrice'].plot(kind='density', title='Sale Price Density',
3                        linewidth=2)
4 plt.show()
5
6 # 多列密度图叠加：对比不同列的分布形状
7 data[['salePrice', 'rentPrice']].plot(kind='density', linewidth=2,
8                                       title='Price Density Comparison')
9 plt.show()
10
11 # 直方图 + 密度图叠加（更直观）
12 data['salePrice'].plot(kind='hist', bins=30, density=True,
13                       alpha=0.5, edgecolor='black')
14 data['salePrice'].plot(kind='density', linewidth=2)
15 plt.title('Histogram + Density Overlay')
16 plt.show()
```

【Example / 实战案例】Hexbin Plot (kind='hexbin') / 六边形分箱图

当散点图因数据量过大而严重重叠时，hexbin 通过六边形分箱 + 颜色深浅表示每个区域内点的密度，是大数据量二维分布的理想选择。

```
1 # Hexbin：用颜色深浅表示点的密集程度
2 data.plot(kind='hexbin', x='rentPrice', y='salePrice',
3                  gridsize=30, title='Price Hexbin Density',
4                  colormap='YlOrRd', sharex=False, figsize=(8, 6))
5 plt.show()
6
7 # Hexbin 与散点图对比：数据量大时 hexbin 明显更清晰
8 # data.plot(kind='scatter', x='rentPrice', y='salePrice', alpha=0.1)
```

```
9 # data.plot(kind='hexbin', x='rentPrice', y='salePrice', gridsize=20)
```